

CS714: Secure Computation

Adithya Vadapalli

IIT Kanpur

February 10, 2026

Secret Sharing Setup

- Let $\mathbb{F}$ be a finite field with $|\mathbb{F}| > n$.
- Secret $s \in \mathbb{F}$ is shared using a random polynomial

$$f(X) = s + a_1X + \cdots + a_tX^t$$

of degree at most t .

- Each player P_i receives share

$$s_i = f(i)$$

- Any t shares reveal *no information* about s .
- Any $t + 1$ shares allow reconstruction of s .

Lagrange Interpolation

Let $h(X)$ be a polynomial of degree at most ℓ .

If $C \subset \mathbb{F}$ with $|C| = \ell + 1$, then

$$h(X) = \sum_{i \in C} h(i) \delta_i(X)$$

where

$$\delta_i(X) = \prod_{\substack{j \in C \\ j \neq i}} \frac{X - j}{i - j}$$

- $\delta_i(j) = 1$ if $i = j$
- $\delta_i(j) = 0$ if $i \neq j$

Why Interpolation Works

- Each $\delta_i(X)$ has degree at most ℓ .
- $\sum_{i \in C} h(i)\delta_i(X)$ matches $h(X)$ on all points in C .
- A nonzero polynomial of degree $\leq \ell$ cannot have more than ℓ roots.
- Therefore,

$$h(X) = \sum_{i \in C} h(i)\delta_i(X)$$

Recombination Vector

A key consequence:

There exist public values $r_1, \dots, r_n$ such that

$$h(0) = \sum_{i=1}^n r_i h(i)$$

- $r_i = \delta_i(0)$
- The vector $(r_1, \dots, r_n)$ is independent of $h(X)$
- Same recombination works for *all* degree $\leq n - 1$ polynomials

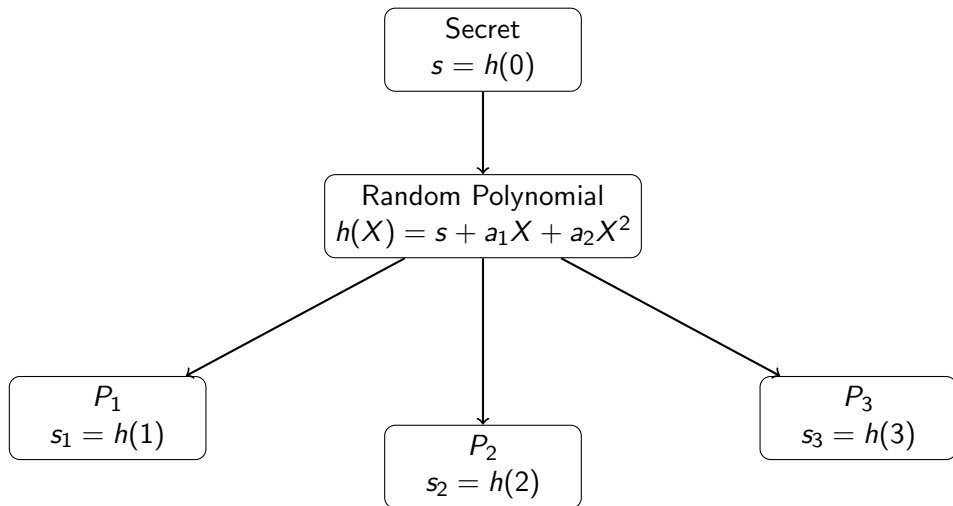
Privacy of Shamir Sharing

- Let $f(X)$ be uniformly random of degree $\leq t$ with $f(0) = s$.
- For any set $C \subset \mathbb{F}$ with $|C| = t$ and $0 \notin C$:

$$(f(i))_{i \in C} \sim \text{Uniform over } \mathbb{F}^t$$

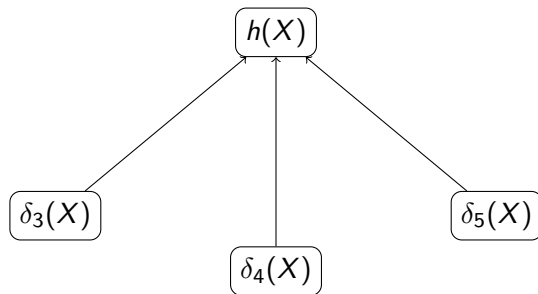
- Distribution is independent of s .
- Hence, t shares give no information about the secret.

Shamir Secret Sharing



Threshold $t = 2$: any 3 shares reconstruct s , any 2 give no information.

Lagrange Interpolation



$$h(X) = \sum_{i \in C} h(i) \delta_i(X), \quad \delta_i(X) = \prod_{j \in C, j \neq i} \frac{X - j}{i - j}$$

Worked Example: Setup

- Field: $\mathbb{F}_{11}$
- Threshold: $t = 2$
- Polynomial:

$$h(X) = s + a_1X + a_2X^2$$

- Choose:

$$s = 7, \quad a_1 = 4, \quad a_2 = 1$$

- Thus:

$$h(X) = 7 + 4X + X^2$$

Share Generation

Compute shares:

$$h(1) = 7 + 4 + 1 = 12 \equiv 1 \pmod{11}$$

$$h(2) = 7 + 8 + 4 = 19 \equiv 8 \pmod{11}$$

$$h(3) = 7 + 12 + 9 = 28 \equiv 6 \pmod{11}$$

$$h(4) = 7 + 16 + 16 = 39 \equiv 6 \pmod{11}$$

$$h(5) = 7 + 20 + 25 = 52 \equiv 8 \pmod{11}$$

Shares:

$$(1, 1), (2, 8), (3, 6), (4, 6), (5, 8)$$

Lagrange Basis Polynomials

Use shares at points $\{3, 4, 5\}$.

$$\delta_3(X) = \frac{(X-4)(X-5)}{(3-4)(3-5)} = \frac{(X-4)(X-5)}{2}$$

$$\delta_4(X) = \frac{(X-3)(X-5)}{(4-3)(4-5)} = -(X-3)(X-5)$$

$$\delta_5(X) = \frac{(X-3)(X-4)}{(5-3)(5-4)} = \frac{(X-3)(X-4)}{2}$$

All arithmetic is mod 11.

Polynomial Reconstruction

$$h(X) = 6 \delta_3(X) + 6 \delta_4(X) + 8 \delta_5(X)$$

After simplification:

$$h(X) = X^2 + 4X + 7 \pmod{11}$$

Recovered polynomial matches original.

$$s = h(0)$$

$$\delta_3(0) = 10, \quad \delta_4(0) = 7, \quad \delta_5(0) = 6$$

$$s = 10 \cdot 6 + 7 \cdot 6 + 6 \cdot 8 = 150 \equiv 7 \pmod{11}$$

$$\boxed{s = 7}$$

Circuit Evaluation with Passive Security

The protocol evaluates an arithmetic circuit securely against *passive* adversaries.

- Parties: $P_1, \dots, P_n$
- Field: $\mathbb{F}$
- Adversary threshold: $t < n/2$

Circuit Evaluation with Passive Security

The protocol evaluates an arithmetic circuit securely against *passive* adversaries.

- Parties: $P_1, \dots, P_n$
- Field: $\mathbb{F}$
- Adversary threshold: $t < n/2$

Three Phases

- 1 Input sharing
- 2 Computation
- 3 Output reconstruction

Input Sharing Phase

- Each party P_i holds input $x_i \in \mathbb{F}$
- P_i distributes a t -degree sharing

$$[x_i; f_{x_i}]_t$$

where $f_{x_i}(0) = x_i$

- Input gates are now considered processed

Key Invariant

Invariant

Let $a \in \mathbb{F}$ be the value assigned to any wire whose gate has been processed.

Then the players hold a sharing

$$[a; f_a]_t$$

for some degree- t polynomial f_a with $f_a(0) = a$.

Key Invariant

Invariant

Let $a \in \mathbb{F}$ be the value assigned to any wire whose gate has been processed.

Then the players hold a sharing

$$[a; f_a]_t$$

for some degree- t polynomial f_a with $f_a(0) = a$.

- Invariant holds after input sharing
- Protocol ensures it holds after every gate

Computation Phase

- Process gates in computational (topological) order
- For each unprocessed gate:
 - Inputs already satisfy the invariant
 - Locally compute output sharing

Addition Gate

- Inputs: $[a; f_a]_t, [b; f_b]_t$
- Players compute locally:

$$[a; f_a]_t + [b; f_b]_t = [a + b; f_a + f_b]_t$$

- Degree remains t
- Invariant preserved

Multiply-by-Constant Gate

- Input: $[a; f_a]_t$
- Constant: $\alpha \in \mathbb{F}$
- Players compute:

$$\alpha[a; f_a]_t = [\alpha a; \alpha f_a]_t$$

- Degree unchanged
- Invariant preserved

Multiplication Gate: Problem

- Inputs: $[a; f_a]_t, [b; f_b]_t$
- Naive multiplication gives:

$$f_a \cdot f_b$$

- Degree becomes $2t$
- Violates invariant

Multiplication Gate: Problem

- Inputs: $[a; f_a]_t, [b; f_b]_t$
- Naive multiplication gives:

$$f_a \cdot f_b$$

- Degree becomes $2t$
- Violates invariant

Goal

Reduce degree back to t while preserving the value ab

Multiplication Gate: Degree Reduction

- 1 Players compute

$$[a; f_a]_t \cdot [b; f_b]_t = [ab; f_a f_b]_{2t}$$

- 2 Define $h = f_a f_b$

$$h(0) = ab$$

- 3 Each P_i holds $h(i)$ and reshapes it into

$$[h(i); f_i]_t$$

Multiplication Gate: Recombination

- $\deg(h) = 2t \leq n - 1$
- Let $r = (r_1, \dots, r_n)$ be recombination vector

$$h(0) = \sum_{i=1}^n r_i h(i)$$

- Players compute:

$$\sum_i r_i [h(i); f_i]_t = [h(0); \sum_i r_i f_i]_t = [ab; f_{ab}]_t$$

Computation Phase Summary

- All gates processed
- Invariant preserved throughout
- Every wire value is held as a t -sharing

Output Reconstruction

- For each output gate i , players hold:

$$[y_i; f_{y_i}]_t$$

- Each P_j sends $f_{y_i}(j)$ to P_i
- P_i reconstructs:

$$y_i = f_{y_i}(0)$$

using Lagrange interpolation from any $t + 1$ shares

Protocol CEPS: Circuit Evaluation with Passive Security

Setting:

- Parties evaluate an arithmetic circuit over a field $\mathbb{F}$
- Security against *passive (honest-but-curious)* adversaries

Protocol Phases:

① Input Sharing:

- Each input x_i is secret-shared as $[x_i; f_{x_i}]_t$
- All input gates are considered processed

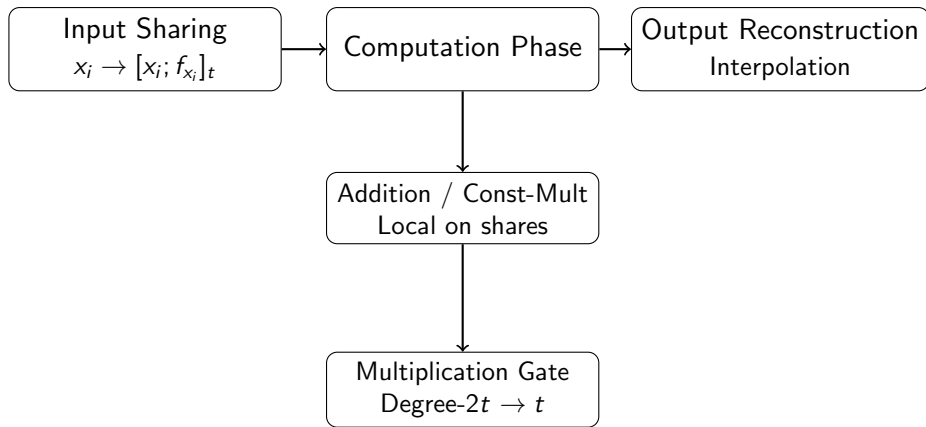
② Computation Phase:

- *Invariant:* Every processed wire with value a is held as $[a; f_a]_t$
- Addition / constant-multiplication: done locally
- Multiplication:
 - Multiply shares $\Rightarrow$ degree- $2t$ polynomial
 - Degree reduction via recombination vector r
 - Resulting sharing preserves value ab

③ Output Reconstruction:

- Parties send shares to output player
- Output recovered via Lagrange interpolation

CEPS Protocol Overview (Passive Security)



Invariant: Every processed wire with value a is held as a sharing $[a; f_a]_t$.

What Remains to Prove

- **Correctness:** Outputs equal circuit evaluation
- **Privacy:** Corrupted parties learn nothing beyond their inputs and outputs

What Remains to Prove

- **Correctness:** Outputs equal circuit evaluation
- **Privacy:** Corrupted parties learn nothing beyond their inputs and outputs

Security Model

Passive (honest-but-curious) adversary corrupting at most $t < n/2$ parties

- Recall the invariant:

Every processed wire with value a is held as $[a; f_a]_t$

- After all gates are processed:
 - Each output wire holds $[y_i; f_{y_i}]_t$
 - $f_{y_i}(0) = y_i$
- Output reconstruction returns exactly y_i

- Recall the invariant:

Every processed wire with value a is held as $[a; f_a]_t$

- After all gates are processed:
 - Each output wire holds $[y_i; f_{y_i}]_t$
 - $f_{y_i}(0) = y_i$
- Output reconstruction returns exactly y_i

Conclusion

Protocol CEPS computes the correct circuit output

What Do Corrupted Parties See?

Corrupted parties receive only two types of messages from honest parties:

Type 1: During Computation

- Shares from input sharing
- Shares sent during multiplication degree-reduction

Type 2: During Output Reconstruction

- Shares of output polynomials $f_{y_i}(j)$

Observation 1: Computation Phase Privacy

- Honest parties' messages are shares of random degree- t polynomials
- Corrupted set has size at most t
- Therefore:
 - They see at most t points on a degree- t polynomial
 - Distribution is information-theoretically uniform

Conclusion

Messages during input sharing and multiplication reveal nothing

Observation 2: Output Reconstruction Privacy

- Corrupted parties already know:
 - Their own shares of f_{y_i}
 - The output value $y_i = f_{y_i}(0)$ (if corrupted)
- Reconstruction gives them $t + 1$ points on f_{y_i}

Key Point

Since $\deg(f_{y_i}) = t$, these values are fully determined by inputs and outputs already known to corrupted parties

Simulation Paradigm

Goal

Construct a simulator S that reproduces the corrupted parties' view using:

- Inputs of corrupted parties
- Outputs of corrupted parties

Simulator Construction (High Level)

The simulator S proceeds as follows:

- ① Run corrupted parties internally on their inputs
- ② Simulate honest-party messages by sampling uniformly random shares
- ③ In output reconstruction:
 - Ensure consistency with known outputs

Simulator Construction (High Level)

The simulator S proceeds as follows:

- ① Run corrupted parties internally on their inputs
- ② Simulate honest-party messages by sampling uniformly random shares
- ③ In output reconstruction:
 - Ensure consistency with known outputs

Result

Simulated view is identically distributed to real execution

Formalizing Observation 1

To formalize privacy, define a stripping function:

Strip(view_j)

Removes from party P_j 's view:

- Shares received from honest parties in output reconstruction

Leaves:

- Random coins
- Messages during computation

Lemma 3.4 (Privacy)

Lemma

Let $C \subseteq \{P_1, \dots, P_n\}$ be corrupted parties, $|C| \leq t$.

Let $x^{(0)}, x^{(1)}$ be two global inputs such that:

$$x_i^{(0)} = x_i^{(1)} \quad \forall i \in C$$

Then:

$$\{\text{Strip}(\text{view}_j^{(0)})\}_{j \in C} \equiv \{\text{Strip}(\text{view}_j^{(1)})\}_{j \in C}$$

Meaning of the Lemma

- Corrupted parties cannot distinguish executions
- As long as their own inputs are unchanged
- Regardless of honest parties' inputs

Lemma 3.4 (Passive Security)

- Let $C \subseteq \{P_1, \dots, P_n\}$ be the set of corrupted parties.
- Let $x^{(0)}, x^{(1)}$ be two input vectors such that $x_i^{(0)} = x_i^{(1)}$ for all $P_i \in C$.
- Claim: The joint view of parties in C is identically distributed in executions on $x^{(0)}$ and $x^{(1)}$.

- We compare the distributions of the views of C in two executions:
 - case $b = 0$: execution on $x^{(0)}$,
 - case $b = 1$: execution on $x^{(1)}$.
- We analyze each phase of the protocol separately.
- Show that C sees the same distribution in both cases.

Input Sharing Phase

- For each $P_i \in C$:
 - P_i distributes shares of $x_i^{(b)}$.
 - Since $x_i^{(0)} = x_i^{(1)}$, distributions are identical.
- For each $P_j \notin C$:
 - P_j distributes shares of $x_j^{(b)}$.
 - Parties in C see at most t shares.
 - These shares are uniformly random and independent of $x_j^{(b)}$.
- Hence, the view of C is identically distributed for $b = 0, 1$.

Addition Gates

- No messages are sent during addition.
- Local computations only.
- Therefore, nothing is revealed to C .
- Distributions are trivially identical in both cases.

Multiplication Gates

- Assume inductively that inputs to the multiplication gate have identical distributions for $b = 0, 1$.
- Parties locally multiply their shares.
- Honest parties send shares of random values.
- Parties in C see only t shares of each sharing.
- These shares are uniformly random and independent of b .
- Thus, the distribution remains identical.

Output Reconstruction Phase

- All output shares are sent to all parties.
- If $P_j \notin C$:
 - Parties in C see nothing.
- If $P_j \in C$:
 - Parties in C see all shares of y_j .
- Shares sent by honest parties are removed by stripping.
- Remaining shares are computed from prior views.

- In every phase, corrupted parties see the same distribution in executions on $x^{(0)}$ and $x^{(1)}$.
- Therefore, the joint view of C is identically distributed.
- This proves passive security of the protocol.

Output Reconstruction

- Output y is secret-shared.
- Corrupted parties see only their output shares.
- Honest shares are never revealed.

Observation 2 (Informal)

Observation

Once the output y is fixed, the distribution of corrupted parties' views does not depend on honest parties' inputs. In other words, values sent by honest parties to the corrupted parties could have been generated by the corrupted parties, given the values they already know plus the output from the output reconstruction.

Defining Dress

Let $C \subseteq \{P_1, \dots, P_n\}$ with $|C| \leq t$.

Let $\{\text{Strip}(\text{view}_j)\}_{j \in C}$ be the stripped views.

Definition

$\text{Dress}_C(\cdot)$ is defined as follows:

- For each sharing instance, corrupted parties hold at most t shares
- Using these shares, interpolate the unique degree- t polynomial
- Reconstruct the full sharing (all n shares)
- Replace the missing honest-party shares in the view by the reconstructed ones

This yields a *full view* consistent with the corrupted shares.

Observation 2

Let $C \subset \{P_1, \dots, P_n\}$ be a set of corrupted parties and assume for simplicity that $|C| = t$. Let $\mathbf{x}$ be any input vector, let $(y_1, \dots, y_n) = f(\mathbf{x})$

For any execution of Protocol CEPS and any corrupted set C with $|C| \leq t$:

$$\text{Dress}_C(\{\text{Strip}(\text{view}_j)\}_{j \in C}) = \{\text{view}_j\}_{j \in C}.$$

Proof (Part 1: Output Shares)

- Fix a corrupted party $P_j \in C$.
- The view view_j contains n shares of the output value y_j computed during the protocol.
- By correctness of CEPS, the shared value is the correct output $y_j = f_j(x)$.
- Thus, view_j contains a valid t -private sharing of y_j .

Proof (Part 2: Effect of Strip)

- The operation $\text{Strip}(\text{view}_j)$ removes:
 - shares of y_j sent by honest parties.
- It keeps unchanged:
 - shares held by corrupted parties,
 - all other local randomness and messages.
- After stripping, exactly t output shares of y_j remain.

Proof (Part 3: Effect of Dress)

- Dress_{y_C} is given:
 - the stripped views of corrupted parties,
 - the correct outputs $y_C = \{y_j\}_{P_j \in C}$.
- Using the t remaining shares and y_j , it reconstructs the missing shares by interpolation.
- The reconstructed shares are identical to those removed by Strip.

Proof (Conclusion)

- For each $P_j \in C$:

$$\text{Dress}_{y_C}(\text{Strip}(\text{view}_j)) = \text{view}_j.$$

- Applying this argument to all corrupted parties gives the claim.
- Hence, reconstruction leaks no information beyond y_C .

Formalizing the Simulator

- We define a simulator S using the functions Strip and Dress.
- The simulator produces a view for the corrupted parties only.
- Input to the simulator:
 - inputs of corrupted parties,
 - outputs of corrupted parties.

Simulator: Step 1 (Input)

- The input to the simulator is:

$$\{x_j, y_j\}_{P_j \in C}.$$

- Here:
 - x_j is the input of corrupted party P_j ,
 - y_j is the output value received by P_j .

Simulator: Step 2 (Define Global Input)

- Simulator defines a global input vector:

$$x^{(0)} = (x_1^{(0)}, \dots, x_n^{(0)}).$$

- For each party P_j :
 - if $P_j \in C$, set $x_j^{(0)} = x_j$,
 - if $P_j \notin C$, set $x_j^{(0)} = 0$.
- Honest parties are given dummy inputs.

Simulator: Step 3 (Run Protocol)

- Simulator runs Protocol CEPS on input $x^{(0)}$.
- Let $\text{view}_j^{(0)}$ denote the view of party P_j in this execution.
- This execution may use incorrect inputs for honest parties.

Simulator: Step 4 (Define Outputs)

- Simulator defines:

$$y_C = \{y_j\}_{P_j \in C}.$$

- These are the output values provided as simulator input.

Simulator: Step 5 (Dress the Views)

- For each corrupted party P_j :
 - apply Strip to $\text{view}_j^{(0)}$,
 - apply Dress using y_C .
- Simulator outputs:

$$\text{Dress}_{y_C}(\{\text{Strip}(\text{view}_j^{(0)})\}_{P_j \in C}).$$

Why the Simulator Works

- Honest parties may send shares of incorrect values.
- Corrupted parties see at most t shares.
- These shares are uniformly random.
- Strip removes honest-party reconstruction messages.
- Dress replaces them with shares consistent with:
 - corrupted parties' shares,
 - correct output values.

Theorem 3.6 (Simulator Correctness)

- Let $x = (x_1, \dots, x_n)$ be a global input vector.
- Let $(y_1, \dots, y_n) = f(x)$.
- Let $C \subseteq \{P_1, \dots, P_n\}$ with $|C| = t$.
- For each $P_j \in C$, let view_j be its view in a real execution of Protocol CEPS on input x .
- Claim:

$$S(\{x_j, y_j\}_{P_j \in C}) \equiv \{\text{view}_j\}_{P_j \in C}.$$

Interpretation of Theorem 3.6

- Simulator produces a view indistinguishable from real execution.
- Corrupted parties learn nothing beyond:
 - their own inputs,
 - their own outputs.
- This establishes passive security of Protocol CEPS.

Multiplication Setup

- Values $a = f(0)$, $b = g(0)$ are shared using degree-1 polynomials.
- Example:

$$f(x) = 2 + x, \quad g(x) = 3 - x$$

- Goal: compute $c = ab$.

Local Multiplication

- Each party computes:

$$d_i = f(i) \cdot g(i)$$

- Points lie on:

$$d(x) = f(x)g(x)$$

- Degree increases to 2.

Resulting Polynomial

$$d(x) = (2 + x)(3 - x) = 6 + x - x^2$$

- Constant term gives correct output.
- Degree growth must be handled.

Degree Reduction

- Parties reshare d_i using fresh randomness.
- Linear recombination removes high-degree terms.
- Example:

$$c = 3d_1 - 3d_2 + d_3$$

Security of Multiplication

- Fresh randomness masks intermediate values.
- Corrupted parties see uniform shares.
- Only c is revealed.

Why the Bound Matters

- Security proven for $|C| \leq t$.
- What if $|C| > t$?

High-Level Intuition

- Too many shares allow interpolation.
- Honest inputs become reconstructible.
- Privacy is information-theoretically impossible.

Optimality Claim

Theorem

If more than t parties are corrupted, no perfectly private protocol exists.

Two-Party Setting

- Parties P_1, P_2 with inputs $b_1, b_2 \in \{0, 1\}$.
- Compute Boolean function $f(b_1, b_2)$.

Assumptions

Perfect Correctness

Output is always $f(b_1, b_2)$.

Perfect Privacy

Each party's view depends only on its input and the output.

Transcript Sets

- Let $T(b_1, b_2)$ be the set of possible transcripts.
- Privacy implies overlaps.
- Correctness forbids overlaps across outputs.

AND Function Example

$$f(b_1, b_2) = b_1 \wedge b_2$$

- Privacy:

$$T(0, 0) \cap T(0, 1) \neq \emptyset$$

- Privacy:

$$T(0, 1) \cap T(1, 1) \neq \emptyset$$

- Implies:

$$T(0,0) \cap T(1,1) \neq \emptyset$$

- But correctness requires disjointness.
- Contradiction.

Impossibility Result

Theorem

No two-party protocol can compute a nontrivial Boolean function with perfect correctness and perfect privacy.

Big Picture

- Simulation formalizes privacy.
- Multiplication is the core technical challenge.
- Corruption bounds are tight.
- Two parties are fundamentally insufficient.